

Semana 22 - Aula 1

Tópico Principal da Aula: Integração com APIs de Terceiros

Subtítulo/Tema Específico: Estratégias e Ferramentas para Integração Segura

Código da aula: [SIS]ANO2C2B4S22A1

Objetivos da Aula:

- Implementar integração com APIs de terceiros.
- Dominar métodos de autenticação padrão de mercado (OAuth 2.0, JWT, API Keys).
- Aplicar boas práticas de tratamento de respostas, erros e logs em integrações.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet;
- Ambiente de desenvolvimento integrado (IDE) com framework back-end (ex: Spring Boot, Express.js).

Exposição do Conteúdo:

Referência do Slide: Tema 1 - Ferramentas e Padrões de Integração

- **Definição:** A integração com APIs de terceiros é o processo pelo qual um sistema de software se comunica e troca dados com serviços externos, utilizando protocolos como **HTTP/RES**. O uso de ferramentas adequadas simplifica a chamada e a manipulação dos dados.
- **Aprofundamento/Complemento (se necessário):** Em ambientes back-end (como Java/Spring ou Node.js/Express), bibliotecas como **Axios** (para Node.js/JavaScript), **RestTemplate/WebClient** (para Spring) ou **requests** (para Python) são fundamentais. Elas abstraem a complexidade das requisições HTTP e facilitam o gerenciamento de *headers*, *timeouts* e a serialização/desserialização de JSON.
- **Exemplo Prático:** Para integrar um sistema de e-commerce a um serviço de rastreio de encomendas externo, o desenvolvedor utiliza uma ferramenta HTTP (ex: Axios) para enviar o código de rastreio para o endpoint da API da transportadora e recebe um JSON com o status atualizado da entrega.
 - **Link de Vídeo 1 (Ferramentas):** [integrating frontend and backend axios](#)
 - **Link de Vídeo 2 (Padrões REST):** [Boas práticas na Modelagem de API's REST](#)

Referência do Slide: Tema 2 - Métodos de Autenticação (OAuth 2.0, JWT, API Keys)

- **Definição:** A autenticação é o processo de verificação da identidade do cliente (sua aplicação) para garantir que apenas sistemas autorizados possam acessar os recursos da.
 - **API Key:** Uma chave secreta simples, ideal para autenticação de aplicações (client-to-server) onde a identificação do usuário individual não é necessária.

- **OAuth 2.0:** Protocolo de **autorização** (não de autenticação) que permite que um aplicativo acesse recursos de um usuário em um servidor de recursos, sem precisar das credenciais do usuário. Ele delega o acesso via *Access Tokens*.
- **JWT (JSON Web Token):** É o **formato** do token de acesso, não o método de autenticação em si. É um padrão seguro e autocontido (possui informações de identidade e permissões) usado geralmente em conjunto com OAuth 2.0 ou para autenticação *stateless**.
- ***A autenticação *stateless*** (sem estado) é um método de segurança onde o servidor **não armazena nenhuma informação de sessão ou estado de**

login sobre o cliente após o processo inicial de login. Em vez de manter uma sessão ativa no servidor (o que seria uma autenticação *stateful*), a autenticação *stateless* funciona tipicamente assim:

1. **Login Inicial:** O usuário envia suas credenciais (nome de usuário e senha) ao servidor.
2. **Geração do Token:** O servidor verifica as credenciais. Se forem válidas, ele **gera um *token* de acesso** que contém as informações necessárias para identificar o usuário (como o ID do usuário e permissões) e o **assina digitalmente** (para garantir que não foi adulterado).
3. **Envio do Token:** O servidor envia esse *token* de volta para o cliente.
4. **Requisições Subsequentes:** Em todas as requisições futuras, o cliente **inclui o *token*** (geralmente no cabeçalho *Authorization*).
5. **Verificação do Token:** O servidor recebe o *token*, **verifica sua assinatura** (para confirmar sua autenticidade e validade) e, se estiver tudo correto, **concede acesso** ao recurso solicitado.

O servidor pode processar a requisição e autenticar o usuário **apenas com a informação contida no *token***, sem precisar consultar um banco de dados de sessões.

Principais Vantagens

- **Escalabilidade:** É muito mais fácil de escalar, pois qualquer servidor da sua infraestrutura pode autenticar a requisição. Você pode adicionar mais servidores sem se preocupar em compartilhar o estado da sessão entre eles.
- **Performance:** A verificação do *token* é geralmente muito rápida, e não há a sobrecarga de consultas a bancos de dados de sessão em cada requisição.
- **Desenvolvimento Mobile e de APIs:** É o padrão ideal para APIs RESTful, onde o cliente pode ser um aplicativo móvel ou outro serviço.
- **Separação de Preocupações:** O cliente se torna responsável por armazenar o *token* (geralmente em *Local Storage* ou *Cookies HTTP Only*).

Exemplo Comum

O padrão mais popular de autenticação *stateless* é o **JSON Web Token (JWT)**.

A principal diferença é:

- **Stateful:** O servidor guarda um ID de sessão e o cliente envia esse ID (*cookie*) em cada requisição.
- **Stateless:** O servidor não guarda nada; o cliente envia um *token* de dados assinado em cada requisição.
- **Aprofundamento/Complemento (se necessário):** O fluxo mais seguro do OAuth 2.0 é o *Authorization Code Grant* com PKCE, que impede ataques de interceptação de código. O JWT é assinado digitalmente, o que permite que a API verifique se ele foi adulterado, garantindo a integridade dos dados contidos no *payload*.
- **Exemplo Prático:** Ao logar em um site usando o botão "Entrar com Google", o sistema está usando o OAuth 2.0. Seu aplicativo recebe um **Access Token (frequentemente um JWT)** do Google (o *Authorization Server*), que ele usa para acessar seus dados (o *Resource Server*) em seu nome.
 - **Link de Vídeo 1 (OAuth/JWT/API Keys):** [API Authentication EXPLAINED! OAuth vs JWT vs API Keys](#)
 - **Link de Vídeo 2 (OAuth 2.0):** [OAuth 2.0 explained with examples](#)

Referência do Slide: Tema 3 - Tratamento de Respostas (Parsing, Erros e Logs)

- **Definição:** O tratamento de respostas envolve a correta leitura (*parsing*) do corpo da resposta, a identificação de códigos de status HTTP para gerenciar o fluxo da aplicação e a utilização de logs para rastrear a operação.
- **Aprofundamento/Complemento (se necessário):** Para o tratamento de erros, as boas práticas REST exigem o uso adequado dos **Códigos de Status HTTP** (ex: 2xx para sucesso, 4xx para erros do cliente, 5xx para erros do servidor, tabela de erros abaixo). Além disso, o padrão **RFC 7807 (Problem Details for HTTP APIs)** é recomendado para padronizar o corpo das respostas de erro, permitindo que o cliente saiba exatamente o tipo de problema ocorrido.
- **Exemplo Prático:** Quando um usuário tenta cadastrar um email que já existe em um sistema, a API deve retornar um status 409 Conflict (ou 400 Bad Request com detalhe), e não 200 OK com uma mensagem de erro. O log de requisição deve registrar o *traceld* para que a equipe de suporte possa rastrear a falha rapidamente.

Tabela de Códigos de Status HTTP

Categoria	Código	Nome Comum	Descrição Resumida
1xx: Informacional	100	Continue	Indica que a primeira parte do pedido foi recebida e o cliente deve continuar.
	101	Switching Protocols	Indica que o servidor está alterando os protocolos conforme solicitado pelo cliente.
2xx: Sucesso	200	OK	O pedido foi bem-sucedido. O mais comum.
	201	Created	O pedido foi bem-sucedido e um novo recurso foi criado como resultado.
	202	Accepted	O pedido foi aceito para processamento, mas o processamento não foi concluído.
	204	No Content	O servidor processou a solicitação com sucesso, mas não retornará nenhum conteúdo.
3xx: Redirecionamento	301	Moved Permanently	O recurso solicitado foi permanentemente movido para uma nova URL.
	302	Found (Temporarily Moved)	O recurso foi temporariamente movido para uma nova URL.
	304	Not Modified	O recurso não foi modificado desde a última vez que foi acessado pelo cliente.
	307	Temporary Redirect	Redireciona temporariamente o cliente para outra URL, preservando o método HTTP original.
4xx: Erro do Cliente	400	Bad Request	O servidor não conseguiu entender a solicitação devido à sintaxe inválida.
	401	Unauthorized	É necessária autenticação para acessar o recurso.
	403	Forbidden	O cliente não tem permissão de acesso ao recurso solicitado.
	404	Not Found	O recurso solicitado não foi encontrado no servidor. O mais famoso.
	405	Method Not Allowed	O método de requisição usado não é suportado para o recurso.
	408	Request Timeout	O servidor expirou esperando a requisição.
	429	Too Many Requests	O usuário enviou muitas solicitações em um determinado período de tempo (limitação de taxa).
5xx: Erro do Servidor	500	Internal Server Error	Uma condição inesperada impediu o servidor de atender à solicitação.
	501	Not Implemented	O servidor não suporta a funcionalidade necessária para atender à solicitação.

- **Link de Vídeo 1 (Tratamento de Erros):** [Boa prática para modelar respostas com erros em REST APIs \(Problem Details for HTTP APIs\)](#)
- **Link de Vídeo 2 (Parsing/Requisições):** [Como tratar erros em requisições HTTP?](#)

Semana 22 - Aula 2

Tópico Principal da Aula: Utilização de kits de desenvolvimento de software (SDKs)

Subtítulo/Tema Específico: Vantagens e Aplicações Práticas dos SDKs

Código da aula: [SIS]ANO2C2B4S22A2

Objetivos da Aula:

- Utilizar SDKs para facilitar a integração com serviços externos.
- Diferenciar API de SDK, reconhecendo o valor do kit no desenvolvimento.
- Configurar chaves e tokens em ambientes de SDK.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet;
- Documentação oficial do AWS SDK (ou outro SDK de serviço na nuvem).

Exposição do Conteúdo:

Referência do Slide: Tema 1 - Definição e Benefícios dos SDKs

- **Definição:** Um **SDK (Software Development Kit)** é um pacote completo de ferramentas, bibliotecas, documentação, exemplos de código, e, crucialmente, as APIs, que permite a construção de aplicações para uma plataforma ou serviço específico.
- **Aprofundamento/Complemento (se necessário):** Enquanto uma **API** é apenas a interface (o "contrato" de comunicação, ex: `POST /users`), o **SDK** é o kit de ferramentas (a "maleta") que torna o uso da API mais fácil, rápido e seguro. Os SDKs oferecem funções pré-empacotadas que tratam de aspectos complexos como autenticação, tratamento de erros e retries de conexão (repetição de chamadas em caso de falha temporária), reduzindo o tempo de desenvolvimento em até metade.
- **Exemplo Prático:** Para acessar o serviço S3 da AWS, um desenvolvedor pode fazer requisições REST manuais. No entanto, ao usar o **AWS SDK para Java/Python/Node.js**, ele simplesmente chama um método nativo da linguagem (ex: `s3Client.putObject(bucket, key, file)`), e o SDK cuida de todo o processo de autenticação (assinatura da requisição), comunicação e tratamento de exceções.
 - **Link de Vídeo 1 (O que é SDK):** [SDK \(DevKit\) // Programmer's Dictionary](#)
 - **Link de Vídeo 2 (Vantagens do SDK):** [Porque integrar sua aplicação com o AWS SDK? Quais as vantagens de usar o SDK?](#)

Semana 22 - Aula 3

Tópico Principal da Aula: Configuração de Serviços na Nuvem

Subtítulo/Tema Específico: Orquestração de Microserviços e Plataformas Cloud

Código da aula: [SIS]ANO2C2B4S22A3

Objetivos da Aula:

- Configurar serviços essenciais na nuvemANO2C2B4S22A3.pdf].
- Compreender os conceitos de desempenho, escalabilidade e monitoramento em ambientes de nuvem.
- Reconhecer a função do Kubernetes na orquestração de microsserviços.

Recursos Adicionais (Sugestão, pode ser adaptado):

- Caderno para anotações;
- Acesso ao laboratório de informática e/ou internet;
- Conta de teste nas plataformas AWS, Azure ou GCP (Google Cloud Platform).

Exposição do Conteúdo:

Referência do Slide: Tema 1 - Plataformas e Estratégias de Configuração na Nuvem

- **Definição:** A configuração de serviços na nuvem envolve a escolha e o provisionamento de recursos (computação, armazenamento, redes e orquestração) nas principais plataformas do mercado: **AWS (Amazon Web Services), Azure (Microsoft) e GCP (Google Cloud Platform)**..
- **Aprofundamento/Complemento (se necessário):** Embora as plataformas tenham nomes de serviços diferentes (ex: EC2 na AWS, VMs no Azure e Compute Engine no GCP), todas oferecem soluções para **escalabilidade** (aumentar a capacidade de recursos automaticamente), **desempenho** (otimização de latência e processamento) e **monitoramento** (uso de ferramentas nativas como CloudWatch/AWS, Monitor/Azure ou Operations/GCP para diagnóstico e logs).
- **Exemplo Prático:** Para hospedar uma aplicação back-end que usa API de terceiros (Aula 1), o desenvolvedor deve configurar um serviço de computação (como AWS Fargate ou Azure App Service), um banco de dados gerenciado (RDS ou Azure SQL) e um serviço de monitoramento.
 - **Link de Vídeo 1 (Comparativo Cloud):** [Aprenda AWS, AZURE e GCP - Pipeline de dados com Airflow e Containers](#)
 - **Link de Vídeo 2 (Cloud Engineer):** [GCP Associate Cloud Engineer | Aula 8 | Treinamento Oficial](#)

Referência do Slide: Tema 2 - Orquestração com Kubernetes

- **Definição:** Kubernetes é um sistema de código aberto (open-source) para automatizar a implantação, o dimensionamento e o gerenciamento de aplicações **containerizadas** (Docker).
- **Aprofundamento/Complemento (se necessário):** O Kubernetes é crucial para arquiteturas de **microsserviços**, pois lida com a orquestração de containers, garantindo que os serviços estejam sempre disponíveis (*self-healing*), balanceando a carga de tráfego (*load balancing*) e facilitando as implantações sem *downtime*. As três grandes nuvens oferecem serviços gerenciados de Kubernetes: **EKS** (AWS), **AKS** (Azure) e **GKE** (Google Cloud).

- **Exemplo Prático:** Um aplicativo de e-commerce é composto por microsserviços de Pagamento, Inventário e Usuários. O Kubernetes gerencia a implantação desses três serviços em containers separados. Se o serviço de Pagamento receber um pico de tráfego, o Kubernetes o escala automaticamente, aumentando o número de réplicas do container para manter o desempenho.
 - **Link de Vídeo 1 (Kubernetes no Azure):** [Tudo que Preciso para Iniciar com Kubernetes e AKS](#)
 - **Link de Vídeo 2 (Kubernetes na AWS):** [Executando Kubernetes com a AWS - Português](#)